

Reinforcement Learning Summative Assignment Report

Student Name: Mugabo Patricia

Video Recording: [ADD VIDEO URL BEFORE SUBMISSION]

GitHub Repository: [ADD GITHUB URL BEFORE SUBMISSION]

1. Project Overview

AquaForecast is a mission-based reinforcement-learning extension of a water-demand forecasting and decision-support platform for Kimironko Sector, Gasabo District, Kigali. The simulated agent recommends a daily operating strategy that balances service reliability, reservoir resilience, treatment limits, leakage, energy use and uncertain weather-driven demand. DQN, REINFORCE, PPO and A2C interact with the same stochastic Gymnasium environment for objective comparison. Recommendations remain subject to human operator approval and are not sent directly to infrastructure.

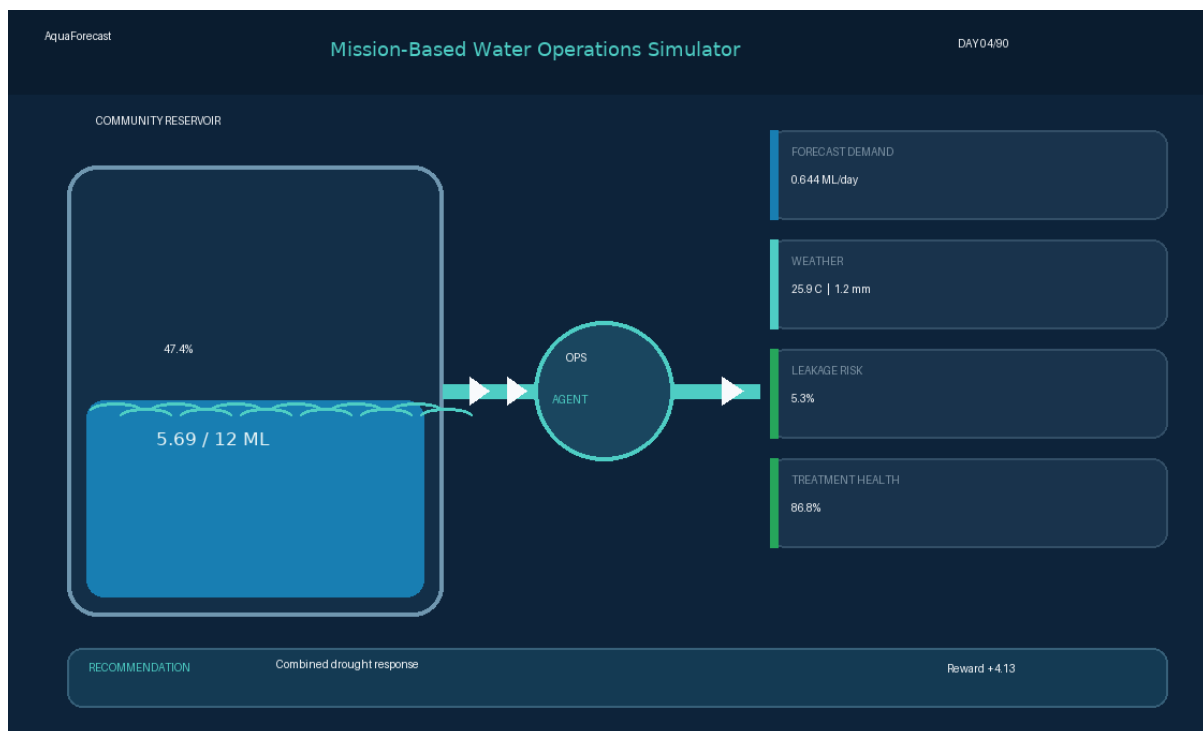


Figure 1. OpenGL-backed AquaForecast dashboard showing reservoir status, operational observations and the agent recommendation.

2. Environment Description

2.1 Agent, start state and dynamics

The agent represents a human-in-the-loop operational planner. Each step equals one day. At reset, the simulator randomizes day of year, 5.4–9.6 ML reservoir storage, recent demand, treatment and pipe health, and 1.2–2.5 ML strategic reserve. Scenario options modify these values for drought, demand surge, high leakage, plant outage or mixed stress. Weather, demand, forecast error, tariff, leakage and equipment health then evolve stochastically.

2.2 Discrete action space

ID	Action	Real-world mapping
0	Maintain forecast plan	Forecast-based production and monitoring
1	Reduce production 20%	Save energy when shortage risk is low
2	Increase production 20%	Prepare for moderate peak demand
3	Increase production 40%	Respond to high demand or low storage
4	Release strategic reserve	Controlled 0.16 ML release; reserve is finite
5	Activate backup supply	High-cost 0.13 ML borehole/tanker equivalent
6	Issue conservation advisory	Five-day voluntary demand reduction
7	Dispatch leak-response crew	Immediate leak reduction and pipe-health gain
8	Combined drought response	Production increase plus seven-day advisory
9	Preventive maintenance	Temporary throughput loss; long-term health gain

2.3 Terminal conditions

An episode is truncated after 90 simulated days. It terminates early after three consecutive days with more than 30% unmet demand, representing unacceptable service failure. The tests also cover every action, observation bounds, seeded reset behavior, time-limit behavior and frontend JSON serialization.

2.4 Observation space

Observation	Meaning	Real source	Type	Physical range
Reservoir level	Stored treated water	Level/pressure sensor	float32	0–12 ML
Forecast demand	Next-day requirement	AquaForecast ML/API	float32	0.20–0.95 ML
Previous demand	Most recent use	Smart meters/dataset	float32	0.25–0.95 ML
Rainfall forecast	Expected rainfall	Weather API	float32	0–25 mm
Temperature	Demand driver	Weather API/sensor	float32	17–35 C
Humidity	Weather context	Weather API/sensor	float32	25–95%
Dry-season index	Seasonal water stress	Calendar/climate data	float32	0–1
Leakage risk	Distribution loss risk	Flow/pressure sensors	float32	2.5–18%
Treatment health	Available plant capacity	SCADA/maintenance	float32	45–100%
Energy tariff	Production cost signal	Utility API	float32	0.40–1.00 norm.
Forecast uncertainty	Prediction confidence	ML model output	float32	4–25%
Conservation active	Advisory time remaining	Campaign database	float32	0–7 days
Maintenance benefit	Health benefit remaining	Maintenance system	float32	0–14 days
Strategic reserve	Emergency stock available	Inventory/level sensor	float32	0–2.5 ML
Day sine	Cyclical season encoding	Calendar	float32	0–1
Day cosine	Cyclical season encoding	Calendar	float32	0–1

2.5 Reward structure

The daily reward is $R = 5S - 15U^2 - 4I(U > 0.10) - 2|L - 0.55| - 1.25E - 6O - C + 0.45B$. Here S is service ratio, U unmet-demand ratio, L normalized reservoir level, E normalized production energy cost, O overflow fraction, C action cost including repeat-advisory fatigue, and B indicates safe storage with at least 98.5% service. This prevents unlimited production, emergency supply and repeated conservation messages from becoming cost-free shortcuts.

3. System Analysis and Design

3.1 Shared comparison protocol

All methods use the same 16-value Box observation, ten-action Discrete space, 90-day horizon, reward function, evaluation seeds and six evaluation scenarios. The policy network uses two 64-unit hidden layers unless the relevant experiment changes hidden width. Best models are selected by mean reward over seeded evaluation episodes, with failure rate used as a safety tie-breaker.

3.2 Deep Q-Network (DQN)

Stable-Baselines3 DQN is the value-based method. It uses an online Q-network, target network, replay buffer, mini-batch temporal-difference updates and epsilon-greedy exploration. Experiments vary learning rate, discount factor, replay capacity, batch size, exploration decay/final epsilon and target update interval.

3.3 REINFORCE

Because Stable-Baselines3 does not ship a REINFORCE class, the required Monte-Carlo policy-gradient algorithm is implemented in PyTorch while retaining the identical Gymnasium environment and evaluation interface. A categorical policy samples actions, discounted returns are normalized, entropy supports exploration and gradients are accumulated across several episodes.

3.4 Proximal Policy Optimization (PPO)

Stable-Baselines3 PPO is an on-policy actor–critic method. It uses generalized advantage estimation, separate policy/value branches and a clipped surrogate objective that limits destabilizing policy updates. Tuning covers learning rate, gamma, rollout length, batch size, GAE lambda, clip range and entropy coefficient.

3.5 Advantage Actor–Critic (A2C)

Stable-Baselines3 A2C jointly optimizes a categorical actor and state-value critic from short on-policy rollouts. Experiments vary learning rate, gamma, rollout length, GAE lambda, entropy coefficient, value-loss weight and maximum gradient norm.

3.6 Product integration

The environment exposes a JSON-serializable payload containing location, timestamp, system measurements, action, explanation and a human-approval notice. This can be returned by an API to an AquaForecast web/mobile dashboard without exposing the training object or granting direct infrastructure control.

4. Hyperparameter Experiments

Each algorithm used ten deliberately different configurations. Stable-Baselines3 methods trained for 25,000 timesteps per run; REINFORCE trained for 250 episodes. Mean reward +/- standard deviation is measured over twelve seeded normal-condition evaluation episodes. Green identifies the promoted model.

4.1 DQN - 10 runs

Run	LR	Gamma	Buffer	Batch	Explore	Final eps	Target	Reward +/- SD
1	1e-04	0.95	5000	32	0.3	0.08	500	405.73 +/- 65.62
2	3e-04	0.97	10000	64	0.4	0.05	750	393.28 +/- 62.51
3	5e-04	0.99	10000	64	0.5	0.03	1000	392.87 +/- 5.22
4	1e-03	0.99	15000	128	0.35	0.05	500	400.00 +/- 61.57
5	5e-05	0.995	15000	128	0.7	0.02	1500	220.49 +/- 100.76
6	2e-04	0.9	5000	32	0.2	0.1	250	402.52 +/- 79.63
7	7e-04	0.97	12500	64	0.6	0.01	1000	414.75 +/- 62.40
8	1e-04	0.999	20000	256	0.55	0.04	2000	420.20 +/- 66.29
9	3e-04	0.93	7500	64	0.25	0.15	400	413.50 +/- 64.47
10	8e-04	0.995	17500	128	0.65	0.02	1250	411.79 +/- 28.53

DQN run 8 achieved the largest normal-condition mean (420.20), although its SD was 66.29. High gamma (0.999), a 20,000-transition buffer and slower target updates supported long-horizon planning. Run 3 was most stable (SD 5.22), while the smallest learning rate in run 5 learned too slowly and scored 220.49.

4.2 REINFORCE - 10 runs

Run	LR	Gamma	Hidden	Entropy	Batch eps	Reward +/- SD
1	1e-04	0.95	64	0.0	4	342.88 +/- 83.05
2	3e-04	0.97	64	0.005	4	400.43 +/- 62.36
3	5e-04	0.99	64	0.01	8	249.08 +/- 119.79
4	1e-03	0.99	128	0.01	8	316.11 +/- 18.57
5	5e-05	0.995	128	0.02	12	397.50 +/- 78.16
6	2e-04	0.9	32	0.0	4	346.28 +/- 29.14
7	7e-04	0.97	128	0.03	8	347.28 +/- 30.50
8	1e-04	0.999	256	0.015	12	231.21 +/- 116.68
9	3e-04	0.93	32	0.04	6	229.75 +/- 117.67
10	8e-04	0.995	64	0.005	10	392.69 +/- 5.04

REINFORCE run 2 produced the best mean (400.43) with LR 3e-4, gamma 0.97 and light entropy regularization. Run 10 was slightly lower (392.69) but far more consistent (SD 5.04). Large networks, very high gamma or strong entropy did not improve the 250-episode budget.

4.3 PPO - 10 runs

Run	LR	Gamma	Steps	Batch	GAE	Clip	Entropy	Reward +/- SD
1	1e-04	0.95	128	32	0.9	0.1	0.0	405.73 +/- 65.62
2	3e-04	0.97	256	64	0.95	0.2	0.005	414.78 +/- 19.56
3	5e-04	0.99	256	64	0.98	0.2	0.01	418.84 +/- 25.41
4	1e-03	0.99	128	64	0.95	0.3	0.01	403.56 +/- 64.47
5	5e-05	0.995	512	128	0.99	0.1	0.02	244.54 +/- 124.76
6	2e-04	0.9	64	32	0.85	0.25	0.0	338.23 +/- 105.68
7	7e-04	0.97	256	32	0.92	0.15	0.03	394.70 +/- 45.87
8	1e-04	0.999	512	64	0.99	0.2	0.015	231.21 +/- 116.68
9	3e-04	0.93	128	32	0.88	0.3	0.04	342.40 +/- 103.72
10	8e-04	0.995	256	128	0.97	0.12	0.005	402.65 +/- 77.25

PPO run 3 achieved 418.84 +/- 25.41 with zero evaluation failures and only 0.094 ML mean unmet water. The moderate clip (0.20), 256-step rollouts and GAE 0.98 balanced update size and variance. Runs 5 and 8 combined small learning rates with long rollouts and underperformed.

4.4 A2C - 10 runs

Run	LR	Gamma	Steps	GAE	Entropy	VF coef	Grad norm	Reward +/- SD
1	1e-04	0.95	5	0.9	0.0	0.5	0.5	405.73 +/- 65.62
2	3e-04	0.97	10	0.95	0.005	0.5	0.5	371.75 +/- 65.27
3	5e-04	0.99	20	0.98	0.01	0.6	0.5	223.38 +/- 95.90
4	1e-03	0.99	10	0.95	0.01	0.4	0.7	351.57 +/- 23.99
5	5e-05	0.995	40	0.99	0.02	0.7	0.3	341.73 +/- 20.25
6	2e-04	0.9	5	0.85	0.0	0.5	1.0	338.23 +/- 105.68
7	7e-04	0.97	20	0.92	0.03	0.6	0.5	340.39 +/- 105.83
8	1e-04	0.999	40	0.99	0.015	0.8	0.3	332.29 +/- 19.13
9	3e-04	0.93	10	0.88	0.04	0.4	0.7	416.05 +/- 65.66
10	8e-04	0.995	20	0.97	0.005	0.7	0.4	331.30 +/- 115.93

A2C run 9 scored 416.05 with gamma 0.93, ten-step rollouts and the largest tested entropy coefficient (0.04). Run 3 fell to 223.38, indicating sensitivity to the interaction between LR, gamma and rollout length. Because one seed was used per configuration, these are comparative observations rather than isolated causal effects.

5. Results and Discussion

5.1 Cumulative rewards



Figure 2. Cumulative reward for the best run of each method. Episode counts differ slightly because SB3 training stops after the requested timestep budget.

PPO accumulated 115,866.50 reward across 278 logged episodes and A2C accumulated 115,249.85 across 277. DQN reached 113,845.08 across 280 episodes, while REINFORCE reached 102,379.96 across its fixed 250 episodes. The nearly linear profiles show positive full-horizon returns; slope is fairer than final height when episode counts differ. Mean reward per logged episode was PPO 416.79, A2C 416.06, REINFORCE 409.52 and DQN 406.59.

5.2 Learning behavior

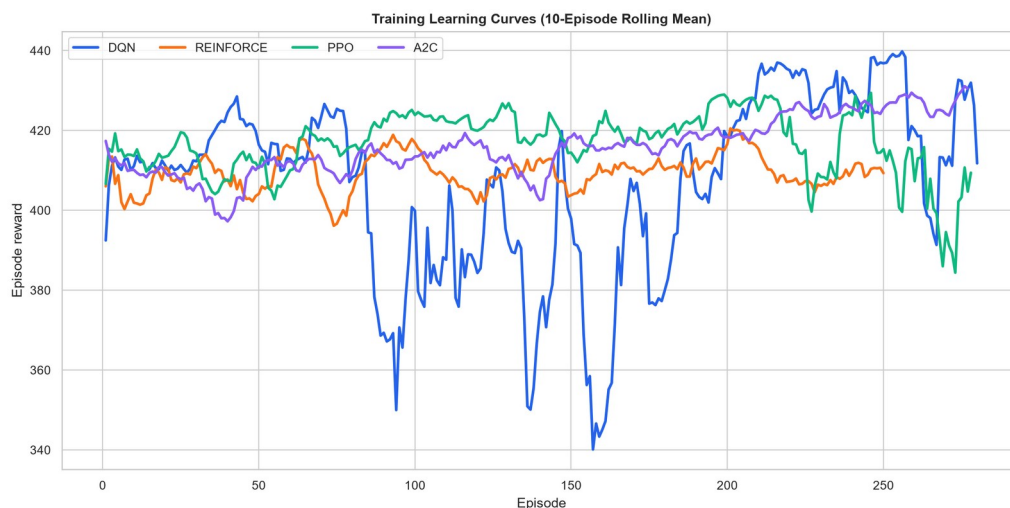


Figure 3. Ten-episode rolling-mean reward during training.

PPO and A2C reached the highest late-training plateau. DQN showed deeper temporary drops, consistent with changing epsilon-greedy data and replay updates. REINFORCE remained smoother but plateaued lower, illustrating a stability-versus-asymptotic-performance trade-off.

5.3 Objective and policy entropy

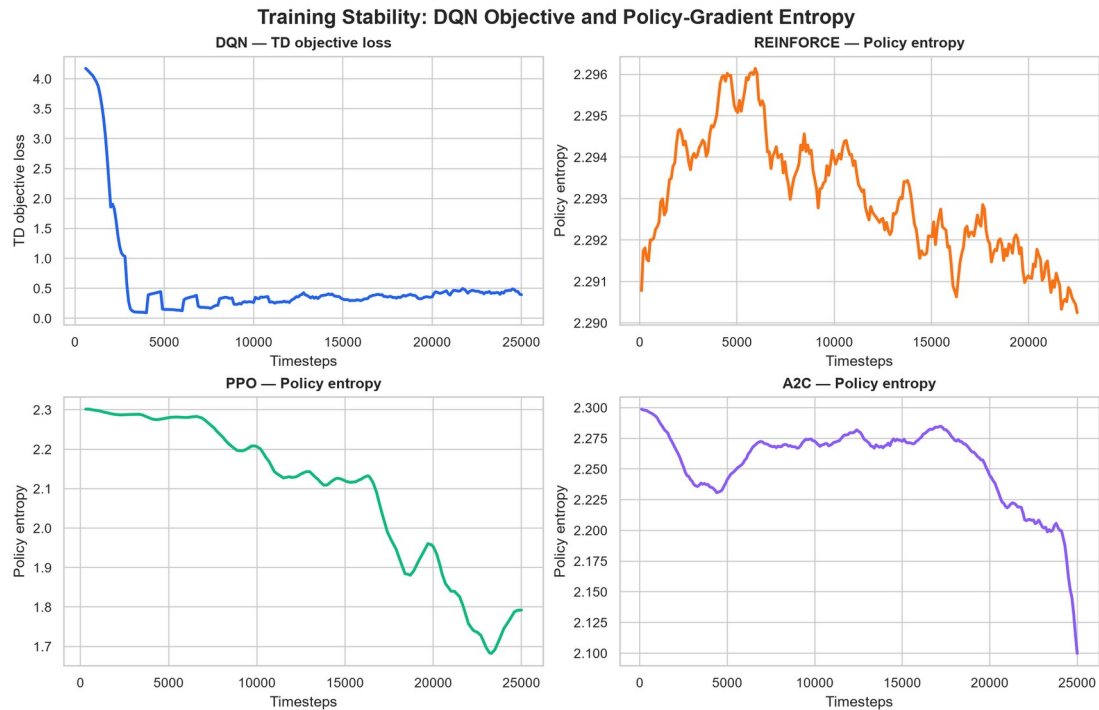


Figure 4. DQN TD-objective loss and policy entropy for the three policy-gradient methods.

DQN's smoothed TD loss decreased from about 4.17 to 0.33, with later oscillations showing continued replay-based correction rather than divergence. REINFORCE entropy stayed near 2.29 (close to $\log(10)=2.303$), so its policy remained broadly exploratory. PPO entropy declined from about 2.30 to 1.79 and A2C to 2.05, indicating increasingly selective policies without complete entropy collapse.

5.4 Episodes to converge



Figure 5. First stable-performance episode; lower is faster.

Convergence is the first 20-episode rolling mean within 2% of that run's peak for at least eight of the next ten windows. REINFORCE met this rule at episode 25, PPO at 91, A2C at 211 and DQN at 215. REINFORCE therefore stabilized fastest, but its lower final/generalization reward shows that rapid convergence can mean an early plateau rather than the best policy.

5.5 Generalization

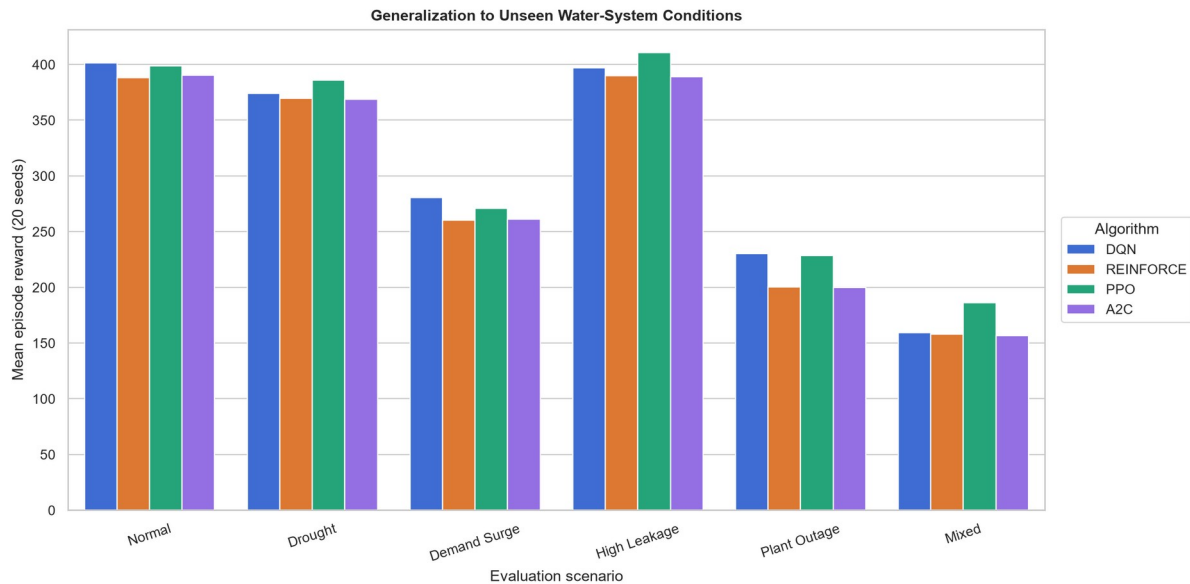


Figure 6. Mean reward across 20 unseen seeds per operational scenario.

Method	Best normal reward	Conv. ep.	6-scenario reward	Unmet ML	Failure
DQN	420.20 +/- 66.29	215	307.04	0.997	47.5%
REINFORCE	400.43 +/- 62.36	25	294.35	1.081	56.7%
PPO	418.84 +/- 25.41	91	313.54	1.268	38.3%
A2C	416.05 +/- 65.66	211	294.31	1.084	57.5%

PPO had the best six-scenario mean reward (313.54) and lowest average failure rate (38.3%), while DQN had the lowest average unmet water (0.997 ML). Drought and high leakage were handled relatively well; demand surge, plant outage and mixed stress reduced every method's reward. PPO's mixed-stress reward was highest (185.96) but its cumulative unmet water was 3.553 ML because it survived longer, whereas DQN terminated more often and recorded 2.016 ML. This exposes a limitation of total-return comparison when episode lengths vary.

6. Conclusion and Discussion

PPO is the recommended demonstration agent because it combined near-best normal reward, lower variability, faster convergence than DQN/A2C, the best average stress-scenario reward and the lowest failure rate. DQN is a strong alternative when minimizing unmet volume is prioritized. REINFORCE was simple and fastest to stabilize but plateaued lower; A2C matched PPO's normal mean less reliably and generalized similarly to REINFORCE. These conclusions are limited by simulated dynamics, one training seed per configuration, reward-weight sensitivity and total reward's dependence on episode length. Future work should use multi-seed tuning, normalize return by days survived, calibrate with utility data, add constrained/safe RL and test multi-zone allocation. All recommendations must remain human-approved before deployment.

References

Mnih et al. (2015), Human-level control through deep reinforcement learning. Nature 518. Williams (1992), Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning 8. Schulman et al. (2017), Proximal Policy Optimization Algorithms. arXiv:1707.06347. Raffin et al. (2021), Stable-Baselines3: Reliable Reinforcement Learning Implementations. JMLR 22.